

Транзакции и целостность данных

Атомарные операции, rollback, \$transaction() и ограничения PostgreSQL

Результаты занятия

- ✓ Понимать назначение транзакций
- ✓ Объяснять свойства ACID на практических примерах
- ✓ Различать COMMIT и ROLLBACK
- ✓ Использовать nested writes Prisma
- ✓ Использовать `prisma.$transaction([])` для независимых операций
- ✓ Использовать интерактивные транзакции через `prisma.$transaction(async (tx) => ...)`
- ✓ Создавать Order и OrderItem как одну бизнес-операцию
- ✓ Сохранять целостность через PK, FK, UNIQUE, NOT NULL и CHECK
- ✓ Понимать роль referential actions
- ✓ Корректно обрабатывать ошибки ограничений базы данных

Учебные вопросы

- 1** **Что такое транзакция** Атомарность, COMMIT, ROLLBACK и ACID
- 2** **Транзакции в Prisma** Nested writes, `$transaction([])` и interactive transactions
- 3** **Транзакция создания заказа** Order + OrderItem как единая бизнес-операция
- 4** **Целостность данных** PK, FK, UNIQUE, NOT NULL, CHECK и referential actions
- 5** **Ошибки и конкурентный доступ** Constraint errors, конфликты и короткие транзакции



Учебный вопрос 1. Что такое транзакция

Несколько операций должны завершиться как единое целое

Проблема нескольких связанных операций

- Создание заказа затрагивает таблицу `orders`.
- Каждая позиция заказа создаётся в `order_items`.
- Операции логически относятся к одному действию пользователя.
- Если часть записей создастся, а часть завершится ошибкой, база окажется в некорректном состоянии.
- Транзакция объединяет несколько запросов в одну неделимую операцию.

Определение транзакции

- Транзакция — последовательность операций с базой данных, которая либо полностью фиксируется, либо полностью отменяется.
- Успех всех операций приводит к COMMIT.
- Ошибка приводит к ROLLBACK.
- Промежуточное некорректное состояние не должно становиться постоянным.
- Транзакции особенно важны при нескольких связанных INSERT, UPDATE и DELETE.

ACID простыми словами

Свойство	Смысл	Пример Internet Shop
Atomicity	Всё или ничего	Order и все OrderItem создаются вместе
Consistency	Правила данных сохраняются	Нельзя создать позицию с несуществующим productId
Isolation	Параллельные операции не должны ломать друг друга	Два запроса не должны создавать противоречивое состояние
Durability	После COMMIT данные не теряются	Созданный заказ остаётся в PostgreSQL после ответа API

Атомарность создания заказа: COMMIT или ROLLBACK



Одна транзакция.
Один результат.
Либо всё, либо ничего.

Надёжные приложения начинаются с транзакций

Транзакция = всё или ничего

✓ Успешная транзакция — COMMIT

- 1  HTTP-клиент отправляет запрос создания заказа
POST /orders
- 2  Создаётся Order
`prisma.order.create(...)`
- 3  Создаются OrderItem (несколько позиций)
`prisma.orderItem.create(...)`
- 4  Все шаги выполнены успешно



COMMIT

Фиксируем все изменения в базе



В базе данных сохранено:



Order
(1 запись)

OrderItem
(n записей)



Все изменения фиксируются

✗ Неуспешная транзакция — ROLLBACK

- 1  HTTP-клиент отправляет запрос создания заказа
POST /orders
- 2  Создаётся Order
`prisma.order.create(...)`
- 3  Создаётся первая позиция OrderItem
`prisma.orderItem.create(...)`
- 4  Ошибка на следующей позиции
Например, несуществующий `productId` или ошибка проверки данных



ROLLBACK

Отменяем все изменения транзакции



В базе данных ничего не сохранено:



Order
(нет записей)

OrderItem
(нет записей)



Отменяются все изменения транзакции

COMMIT и ROLLBACK

COMMIT

- Все проверки прошли
- Все запросы выполнились
- Изменения фиксируются
- Другие операции видят итоговое состояние

ROLLBACK

- Возникла ошибка
- Нарушено бизнес-правило
- Сработало ограничение БД
- Все изменения транзакции отменяются

Границы транзакции

1

Начало

Приложение начинает логически связанную операцию.

2

Запросы

Внутри выполняются чтения и изменения данных.

3

Проверки

Приложение проверяет бизнес-условия и результаты запросов.

4

COMMIT или ROLLBACK

Успех фиксирует изменения, ошибка отменяет их.



Учебный вопрос 2. Транзакции в Prisma

Выбираем механизм под тип задачи

Способы транзакционной работы в Prisma

Сценарий	Механизм	Когда использовать
Связанные зависимые записи	Nested writes	Создать родителя и дочерние записи вместе
Независимые операции	<code>\$transaction([])</code>	Несколько запросов не зависят от результатов друг друга
Read → check → write	Interactive transaction	Нужна логика между запросами

Nested writes

- Nested write позволяет создать или изменить связанные записи внутри одной операции Prisma.
- Подходит для зависимых записей.
- Связанные изменения выполняются атомарно.
- При ошибке вся операция откатывается.
- Удобен, когда все данные известны до выполнения запроса.

Пример nested write

JAVASCRIPT

```
const order = await prisma.order.create({
  data: {
    userId: 1,
    status: "new",
    items: {
      create: [
        { productId: 1, quantity: 2, price: 420000 },
        { productId: 2, quantity: 1, price: 35000 }
      ]
    }
  },
  include: { items: true }
});
```

Что важно

- Order создаётся вместе с OrderItem.
- items.create содержит дочерние записи.
- Если одна позиция нарушит ограничение, операция не должна сохраниться частично.
- Этот вариант удобен, когда цена и все значения уже надёжно определены сервером.

\$transaction([]): независимые операции

JAVASCRIPT

```
const [productCount, orderCount] = await prisma.$transaction([
  prisma.product.count(),
  prisma.order.count()
]);
```

Что важно

- В массив передаются Prisma-запросы.
- Операции выполняются последовательно внутри одной транзакции.
- Если одна операция завершается ошибкой, транзакция откатывается.
- Лучше всего подходит для операций, которые не используют результат предыдущего запроса.

Когда массив \$transaction([]) не подходит

Подходит

- Независимые запросы
- Batch-операции
- Несколько count/findMany
- Операции с заранее известными id

Неудобен

- Нужно получить id и использовать его дальше
- Нужно выполнить условную проверку
- Нужно остановиться по бизнес-правилу
- Нужна сложная последовательность read-modify-write

Interactive transaction

- Интерактивная транзакция передаёт callback в `prisma.$transaction()` и предоставляет транзакционный клиент `tx`.
- Внутри callback можно выполнять несколько запросов.
- Результаты предыдущего запроса можно использовать дальше.
- Можно выполнять условия и бросать исключения.
- Любая ошибка внутри callback приводит к rollback.

Базовый шаблон interactive transaction

JAVASCRIPT

```
const result = await prisma.$transaction(async (tx) => {  
  const order = await tx.order.create({  
    data: { userId: 1, status: "new" }  
  });  
  
  await tx.orderItem.create({  
    data: {  
      orderId: order.id,  
      productId: 1,  
      quantity: 2,  
      price: 420000  
    }  
  });  
  
  return order;  
});
```

Что важно

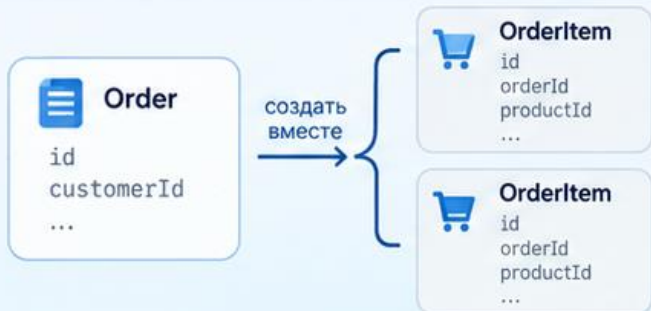
- Все запросы выполняются через tx.
- order.id доступен следующему запросу.
- При успешном завершении callback происходит commit.
- Если callback выбросит ошибку, Prisma выполняет rollback.

Nested writes vs \$transaction([]) vs interactive transaction

Три способа работы с транзакциями в Prisma — для разных задач.

Nested writes

Одна операция
для связанных данных



```
prisma.order.create({
  data: {
    ...,
    items: { create: [...] }
  }
})
```

Когда использовать

- ✓ создание связанных записей
- ✓ все данные известны заранее
- ✓ одна зависимая операция

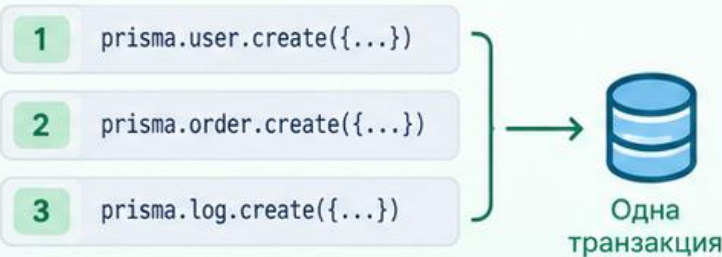


Идея

Связанные записи создаются атомарно внутри одного запроса Prisma.

\$transaction([])

Несколько запросов
в одной транзакции



```
prisma.$transaction([
  prisma.user.create(...),
  prisma.order.create(...),
])
```

Когда использовать

- ✓ независимые операции
- ✓ несколько запросов подряд
- ✓ не нужны результаты предыдущего шага



Идея

Набор независимых запросов выполняется в одной транзакции.

interactive transaction

Гибкая логика
внутри транзакции



```
prisma.$transaction(async (tx) => {
  // ... read
  // ... check
  // ... write
})
```

Когда использовать

- ✓ read → check → write
- ✓ нужны условия и проверки
- ✓ следующий шаг зависит от предыдущего



Идея

Внутри callback можно читать, проверять и изменять данные через tx.

Типичные сценарии



Nested writes

Order + OrderItem[]



\$transaction([])

count + findMany
или несколько независимых запросов



interactive transaction

проверить Product → создать OrderItem



Если возникает ошибка, транзакция откатывается.

Надежные
данные.
Лучшие
приложения.



Учебный вопрос 3. Транзакция создания заказа

Read → validate → create Order → create OrderItem → COMMIT

Бизнес-сценарий POST /api/orders

- Клиент передаёт `userId` и массив `items`.
- Сервер не должен доверять цене из браузера.
- Для каждого `productId` нужно получить `Product` из базы.
- Нужно убедиться, что товар существует и доступен.
- Цена `OrderItem` фиксируется из `Product.price`.
- `Order` и все позиции должны сохраниться атомарно.

Алгоритм создания заказа

- 1 Проверить пользователя** Убедиться, что User существует.
- 2 Создать Order** Получить новый order.id внутри транзакции.
- 3 Проверить товары** Для каждой позиции найти Product и проверить inStock.
- 4 Создать OrderItem** Использовать цену из базы данных.
- 5 Вернуть заказ** При успехе callback завершается и выполняется COMMIT.

Начало createOrder()

JAVASCRIPT

```
export async function createOrder(userId, items) {
  return prisma.$transaction(async (tx) => {
    const user = await tx.user.findUnique({
      where: { id: userId }
    });

    if (!user) {
      throw new Error("User not found");
    }

    const order = await tx.order.create({
      data: { userId, status: "new" }
    });
  });
}
```

Что важно

- Все операции выполняются через tx.
- Пользователь проверяется до создания позиций.
- Ошибка завершит callback и отменит изменения.
- order.id будет использоваться для OrderItem.

Проверка Product и создание OrderItem

JAVASCRIPT

```
for (const item of items) {
  const product = await tx.product.findUnique({
    where: { id: item.productId }
  });

  if (!product || !product.inStock) {
    throw new Error(`Product ${item.productId} is unavailable`);
  }

  await tx.orderItem.create({
    data: {
      orderId: order.id,
      productId: product.id,
      quantity: item.quantity,
      price: product.price
    }
  });
}
```

Что важно

- Product читается внутри той же транзакции.
- Наличие товара проверяется сервером.
- Цена берётся из PostgreSQL, а не из HTTP body.
- Любая ошибка отменяет уже созданный Order.

Возвращаем созданный заказ

JAVASCRIPT

```
return tx.order.findUnique({
  where: { id: order.id },
  include: {
    user: true,
    items: {
      include: {
        product: true
      }
    }
  }
});
```

Что важно

- Финальный запрос также выполняется через tx.
- Service возвращает готовую связанную структуру.
- После успешного завершения callback изменения фиксируются.

Запрос на создание заказа

Запрос: POST /api/orders

Код

```
{
  "userId": 1,
  "items": [
    { "productId": 1, "quantity": 2 },
    { "productId": 2, "quantity": 1 }
  ]
}
```

Результат • JSON

```
{
  "id": 25,
  "status": "new",
  "userId": 1,
  "items": [
    { "productId": 1, "quantity":
2, "price": 420000 },
    { "productId": 2, "quantity":
1, "price": 35000 }
  ]
}
```

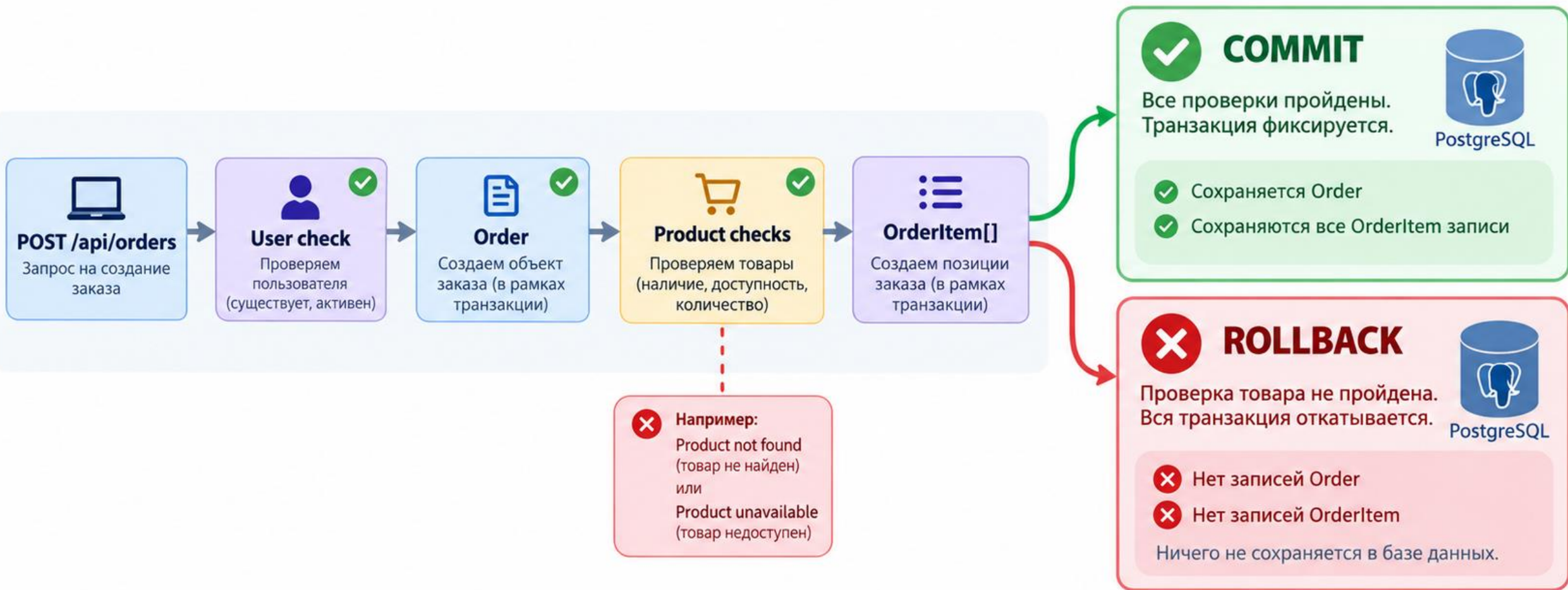
Пояснение

- Цены позиций формирует сервер по данным Product.

Успешная и неуспешная транзакция создания Order

Транзакция — это атомарная операция: либо все изменения сохраняются, либо не сохраняется ничего.

Пример: создание заказа с позициями (Order + OrderItem[]) в одной транзакции



Атомарность транзакции

Order и все OrderItem либо сохраняются вместе, либо не сохраняется ничего.

“Одна операция — один результат: всё или ничего.”

Controller создания заказа

JAVASCRIPT

```
export async function createOrderController(req, res, next) {
  try {
    const order = await createOrder(
      req.body.userId,
      req.body.items
    );

    res.status(201).json(order);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- Controller получает HTTP-данные.
- Service отвечает за транзакцию.
- Controller не управляет COMMIT и ROLLBACK.
- Ошибки передаются централизованному errorHandler.

Как проверить rollback

- Отправьте заказ с двумя позициями.
- Первая позиция должна ссылаться на существующий Product.
- Во второй укажите несуществующий productId.
- Service успеет создать Order внутри транзакции, затем получит ошибку.
- После rollback в таблице orders не должно появиться нового заказа.
- В order_items также не должно остаться частично созданных строк.



Учебный вопрос 4. Целостность данных

Приложение проверяет правила, а база данных защищает инварианты

Три уровня защиты данных

● Validation

Проверяет формат HTTP-запроса до Service.

● Service

Проверяет бизнес-правила приложения.

● PostgreSQL

Гарантирует структурные ограничения и связи.

Основные ограничения целостности

Ограничение	Назначение	Пример
PRIMARY KEY	Уникально идентифицирует строку	products.id
FOREIGN KEY	Не позволяет ссылаться на отсутствующую запись	order_items.product_id
UNIQUE	Запрещает дубликаты уникального значения	users.email
NOT NULL	Запрещает отсутствие обязательного значения	products.name
CHECK	Проверяет условие на уровне PostgreSQL	quantity > 0

PRIMARY KEY и UNIQUE

- @id и @@id отражают первичные ключи.
- @unique и @@unique отражают уникальные ограничения.
- PRIMARY KEY защищает идентичность записи.
- UNIQUE защищает бизнес-уникальность, например email.
- Нарушение UNIQUE отклоняется базой даже при ошибке приложения.

FOREIGN KEY защищает связи

- `Order.userId` должен ссылаться на существующий `User`.
- `OrderItem.orderId` должен ссылаться на существующий `Order`.
- `OrderItem.productId` должен ссылаться на существующий `Product`.
- PostgreSQL отклоняет изменение, нарушающее внешний ключ.
- Prisma relation fields работают поверх этих связей.

Referential actions

Действие	Что происходит
Restrict	Запрещает удаление родительской записи, если есть зависимые записи
Cascade	Удаляет или обновляет связанные записи вместе с родительской
SetNull	Устанавливает внешний ключ в NULL для optional relation
NoAction	Поведение проверки оставляется базе данных
SetDefault	Устанавливает значение внешнего ключа по умолчанию

Явная стратегия удаления Order → OrderItem

PRISMA

```
model OrderItem {
  orderId  Int
  productId Int
  quantity Int
  price    Int

  order  Order @relation(fields: [orderId], references: [id], onDelete: Cascade)
  product Product @relation(fields: [productId], references: [id], onDelete: Restrict)

  @@id([orderId, productId])
}
```

Что важно

- Удаление Order может автоматически удалить его позиции.
- Удаление Product можно запретить, если он используется в заказах.
- Выбор referential action должен соответствовать бизнес-правилу.
- После изменения relation требуется миграция.

Validation → Service rules → Database constraints

Данные защищены тремя слоями — от входящего запроса до сохранения в базе данных.

Входящий HTTP-запрос

```
POST /api/orders  
  
{  
  "userId": 1,  
  "productId": 42,  
  "quantity": 2,  
  "email": "user@example.com"  
}
```

JSON-запрос
от клиента

1 HTTP validation

Проверка входящих данных
(на уровне HTTP / контроллеров)

- ✓ Обязательные поля (не отсутствуют)
- ✓ Корректные типы данных (число, строка и т. д.)
- ✓ Непустые значения (не пустые строки)
- ✓ Положительное количество (quantity > 0)
- ✓ Корректный формат email (user@example.com)

! Некорректные данные отклоняются до выполнения бизнес-логики.

2 Service rules

Проверка бизнес-правил
(на уровне сервисов)

- ✓ Товар существует
- ✓ Пользователь существует
- ✓ Товар есть в наличии
- ✓ Заказ не может быть пустым
- ✓ Цена берётся из базы данных, а не от клиента
- ✓ Проверяются дополнительные бизнес-условия (например, лимиты, роли, статусы)

! Если бизнес-правило нарушено, запрос отклоняется и не передаётся в базу данных.

3 Database constraints

Ограничения базы данных
(PostgreSQL)



- PRIMARY KEY (PK)**
Уникальный идентификатор записи
- FOREIGN KEY (FK)**
Ссылочная целостность между таблицами
- UNIQUE**
Уникальность значений (например, email)
- NOT NULL**
Запрет на NULL в важных полях
- CHECK**
Дополнительные условия (например, quantity > 0, price > 0)

! Даже если предыдущие слои что-то пропустили, ограничения БД защищают структурную целостность данных.



**Безопасные
данные
сохранены**

Данные успешно
сохранены в базе
данных.



Если слой отклоняет запрос — дальнейшая обработка не выполняется!

Данные не попадают на следующий уровень, что предотвращает ошибки и повышает безопасность.



Три слоя защиты = надёжная целостность данных

Проверяем на входе. Контролируем по бизнес-правилам.
Гарантируем на уровне базы данных.

CHECK и ограничения PostgreSQL

- Некоторые ограничения существуют непосредственно на уровне PostgreSQL.
- Например, `quantity > 0` или `price > 0`.
- Такие ограничения защищают данные независимо от Express и Prisma.
- Если схема Prisma не выражает конкретное ограничение, оно может сохраняться в SQL миграции.
- Приложение всё равно должно проверять пользовательский ввод раньше.

Полезные коды ошибок Prisma ORM 7

Код	Ситуация	Пример
P2002	Нарушено UNIQUE	Повторный email
P2003	Нарушен FOREIGN KEY	Несуществующий productId
P2011	Нарушен NOT NULL	Обязательное поле стало NULL
P2025	Требуемая запись не найдена	update/delete отсутствующего ресурса
P2034	Конфликт записи или deadlock	Транзакцию следует повторить



Учебный вопрос 5. Ошибки и конкурентный доступ

Транзакция должна быть короткой и предсказуемой

Не держите транзакцию открытой долго

- Транзакция удерживает соединение с базой.
- Долгие транзакции ухудшают производительность.
- Они увеличивают вероятность блокировок и deadlock.
- Не выполняйте внутри транзакции медленные внешние HTTP-запросы.
- Проверки и запросы должны завершаться как можно быстрее.

Конкурентные изменения

- Несколько пользователей могут одновременно изменять связанные данные.
- Уровень изоляции определяет, какие изменения видит транзакция.
- PostgreSQL и Prisma поддерживают управление изоляцией транзакций.
- Для конфликтных операций может потребоваться Serializable и повтор транзакции.
- На базовом уровне важно понимать сам риск конкурентного доступа.

Пример задания уровня изоляции

JAVASCRIPT

```
const result = await prisma.$transaction(  
  async (tx) => {  
    // связанные операции  
  },  
  {  
    isolationLevel: "Serializable"  
  }  
);
```

Что важно

- Serializable обеспечивает строгую изоляцию.
- При конфликте транзакция может завершиться ошибкой.
- Такие операции обычно выполняют с логикой повторной попытки.
- Для большинства учебных CRUD-сценариев уровень изоляции вручную менять не требуется.

Транзакция не заменяет validation

- Транзакция отвечает за атомарность.
- Validation отвечает за корректность HTTP-входа.
- Service отвечает за бизнес-правила.
- Constraints отвечают за структурную целостность.
- Надёжное приложение использует все уровни одновременно.

Мини-практика: намеренно вызовите rollback

Создайте заказ из двух позиций так, чтобы вторая позиция ссылалась на отсутствующий Product.

Задание

- Запомнить количество строк в orders
- Отправить POST /api/orders
- Использовать существующий productId в первой позиции
- Использовать отсутствующий productId во второй позиции
- Получить ошибку API
- Повторно проверить таблицы orders и order_items

Критерии проверки

- Новый Order не сохранён
- OrderItem первой позиции не сохранён
- База не содержит частичного результата
- Студент может объяснить причину rollback

Что выбрать в типичных ситуациях

Задача	Решение
Создать Order и заранее известные OrderItem	Nested write
Выполнить несколько независимых запросов вместе	\$transaction([])
Прочитать Product, проверить правило и затем записать OrderItem	Interactive transaction
Защитить связь между таблицами	FOREIGN KEY
Запретить дублирующий email	UNIQUE
Отменить все изменения при ошибке	Transaction rollback

От HTTP-запроса до сохранённых данных. Все операции выполняются в транзакции. При любой ошибке — ROLLBACK.

1

Успешное создание заказа (COMMIT)

Все проверки пройдены, данные сохранены.



HTTP Request

POST /api/orders

```
{
  "userId": 1,
  "items": [
    { "productId": 2, "quantity": 1 },
    { "productId": 3, "quantity": 2 }
  ]
}
```



Controller

- Проверяет формат данных
- Вызывает сервис
- Обрабатывает ошибки
- Возвращает ответ

Пример (Express)

```
router.post('/api/orders',
  orderController.create
);
```



Order Service

- Бизнес-логика
- Запуск транзакции
- Проверки пользователей и товаров
- Создание заказа и позиций

```
await prisma.$transaction(
  async (tx) => {
    // ... операции
  }
);
```



Interactive Transaction

Все операции выполняются атомарно

- 1 Проверка пользователя
User exists?
- 2 Проверка товаров
Product exists? Stock $\geq$ quantity?
- 3 Создание Order
tx.order.create(...)
- 4 Создание OrderItem[]
tx.orderItem.createMany(...)



PostgreSQL

Данные сохранены

users	✓
products	✓
orders	✓
order_items	✓

COMMIT

Транзакция успешно завершена



Response

201 Created

```
{
  "id": 101,
  "userId": 1,
  "items": [
    { "id": 1, "productId": 2,
      "quantity": 1 },
    { "id": 2, "productId": 3,
      "quantity": 2 }
  ],
  "total": 67000,
  "status": "CREATED"
}
```

2

Ошибка при создании заказа (ROLLBACK)

Любая ошибка на любом этапе — все изменения отменяются.



HTTP Request

POST /api/orders

```
{
  "userId": 1,
  "items": [
    { "productId": 2, "quantity": 1 },
    { "productId": 999, "quantity": 2 }
  ]
}
```



Controller

- Проверяет формат данных
- Вызывает сервис
- Обрабатывает ошибки
- Возвращает ответ

Пример (Express)

```
try {
  await orderService.create(data)
} catch (e) {
  res.status(400).json({ error })
}
```



Order Service

- Бизнес-логика
- Запуск транзакции
- Проверки пользователей и товаров
- Создание заказа и позиций

```
await prisma.$transaction(
  async (tx) => {
    // ... операции
  }
);
```



Interactive Transaction

Ошибка на одном из этапов

- 1 Проверка пользователя
User exists? ✓
- 2 Проверка товаров
Product with id = 999 not found!
- 3 Создание Order
(не выполняется)
- 4 Создание OrderItem[]
(не выполняется)



PostgreSQL

Изменения не сохранены

users	—
products	—
orders	—
order_items	—

ROLLBACK

Все изменения отменены



Response

400 Bad Request

```
{
  "error": "Product not found",
  "productId": 999,
  "message": "Товар с указанным id не найден"
}
```



Почему это безопасно?

- ✓ Все операции выполняются в одной транзакции
- ✓ При любой ошибке выполняется ROLLBACK
- ✓ База данных остаётся в согласованном состоянии
- ✓ Клиент получает понятный ответ об ошибке



Что сохраняется при успехе?

- Новый заказ в таблице orders
- Позиции заказа в order_items
- Связь с существующими пользователем и товарами
- Все данные сохраняются вместе (атомарно)



Что НЕ сохраняется при ошибке?

- Заказ не создаётся
- Позиции заказа не добавляются
- База данных возвращается к исходному состоянию
- Нет частично сохранённых данных

Контрольные вопросы

1. Что означает атомарность транзакции?
2. Чем COMMIT отличается от ROLLBACK?
3. Какие четыре свойства входят в ACID?
4. Когда удобно использовать nested writes?
5. Для каких операций подходит `$transaction([])`?

6. Зачем `interactive transaction` получает объект `tx`?
7. Почему цену `OrderItem` нельзя принимать из браузера как доверенную?
8. Что произойдёт при `throw` внутри `interactive transaction`?
9. Чем FOREIGN KEY отличается от `validation middleware`?
10. Почему транзакции желательно делать короткими?

Итоги занятия

1

Транзакция объединяет несколько операций в атомарное целое.

2

COMMIT фиксирует успешную транзакцию, ROLLBACK отменяет её.

3

ACID описывает основные свойства надёжных транзакций.

4

Prisma ORM 7 поддерживает nested writes, `$transaction([])` и interactive transactions.

5

Для сценария read → check → write удобно использовать interactive transaction.

6

Все запросы внутри interactive transaction выполняются через tx.

7

Создание Order и OrderItem должно быть атомарным.

8

Целостность защищается одновременно приложением и PostgreSQL.

9

PK, FK, UNIQUE, NOT NULL и CHECK предотвращают некорректные данные.

10

Короткие транзакции снижают риск блокировок и конфликтов.